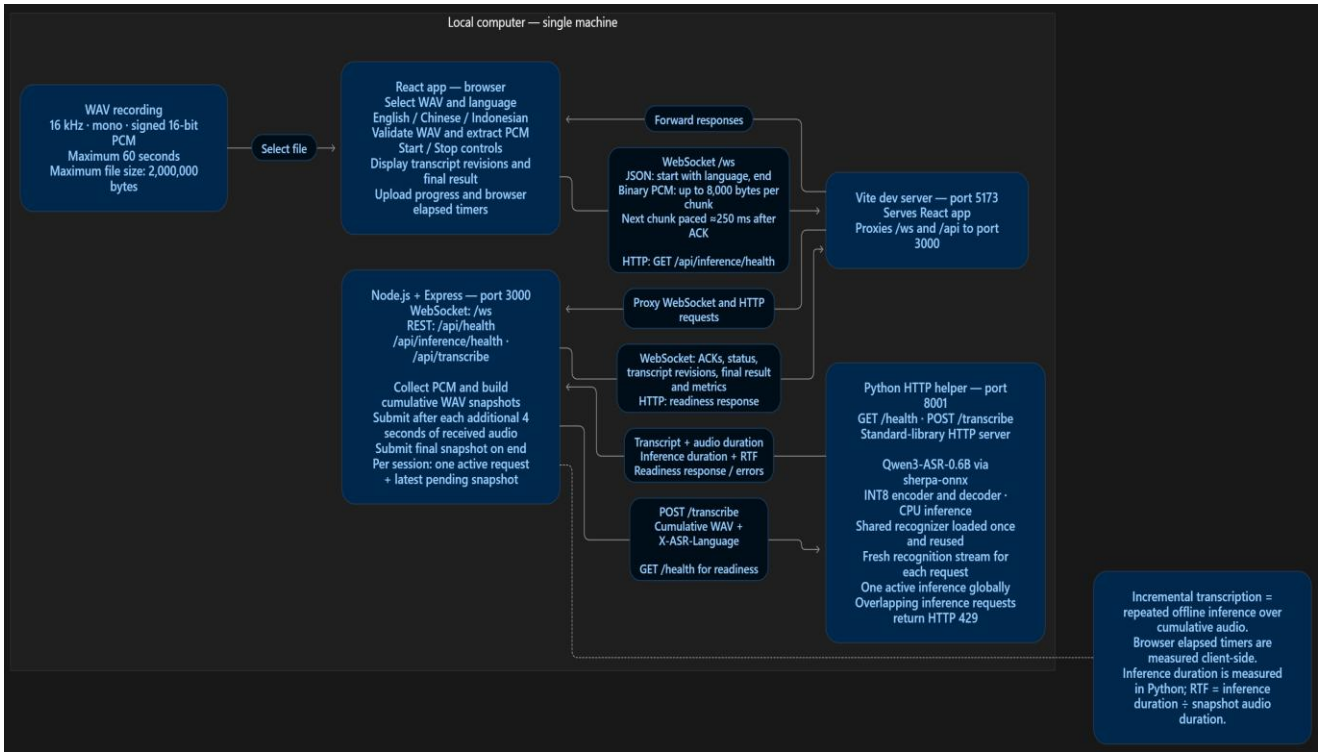

Local computer — single machine
WAV recording
16 kHz · mono · signed 16-bit PCM
Maximum 60 seconds
Maximum file size: 2,000,000 bytes
Select file
React app — browser
Select WAV and language
English / Chinese / Indonesian
Validate WAV and extract PCM
Start / Stop controls
Display transcript revisions and final result
Upload progress and browser elapsed timers
Forward responses
WebSocket /ws
JSON: start with language, end
Binary PCM: up to 8,000 bytes per chunk
Next chunk paced ≈250 ms after ACK
HTTP: GET /api/inference/health
Vite dev server — port 5173
Serves React app
Proxies /ws and /api to port 3000
Node.js + Express — port 3000
WebSocket: /ws
REST: /api/health
/api/inference/health · /api/transcribe
Collect PCM and build cumulative WAV snapshots
Submit after each additional 4 seconds of received audio
Submit final snapshot on end
Per session: one active request + latest pending snapshot
Proxy WebSocket and HTTP requests
WebSocket: ACKs, status, transcript revisions, final result and metrics
HTTP: readiness response
Transcript + audio duration
Inference duration + RTF
Readiness response / errors
POST /transcribe
Cumulative WAV + X-ASR-Language
GET /health for readiness
Python HTTP helper — port 8001
GET /health · POST /transcribe
Standard-library HTTP server
Qwen3-ASR-0.6B via sherpa-onnx
INT8 encoder and decoder · CPU inference
Shared recognizer loaded once and reused
Fresh recognition stream for each request
One active inference globally
Overlapping inference requests return HTTP 429
Incremental transcription = repeated offline inference over cumulative audio.
Browser elapsed timers are measured client-side.
Inference duration is measured in Python; RTF = inference duration ÷ snapshot audio duration.